CIS 635: Assignment 3

Supervised Learning II: Classification with Imbalanced Data

Deadline: Please check the Blackboard post **Total points:** 5

Objective

This assignment provides hands-on practice with **classification**. You will compare **logistic regression**, **decision tree**, and **random forest** models on a highly imbalanced dataset, choose and justify an **evaluation metric** suited to the imbalance, and use that metric to **optimize hyperparameters**. The data are samples; do not make global conclusions from these samples.

Data

- **Credit card transactions:** `https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv`
  Target: `Class` (1 = fraudulent, 0 = legitimate). Features: `Time`, `Amount`, and anonymized principal components `V1`–`V28`.

General rules. Hold out a *stratified* test set (e.g., 80/20) *before* any tuning or resampling, and use it only for final evaluation. Fix a random seed. Fit any scaling or resampling on the training data only (use a `Pipeline`). For every model, report **precision, recall, and F1 for the fraud class** and a **confusion matrix**.

Runtime tip: The dataset has about 285,000 rows. You may tune on a *stratified* sample of the training data (state its size), use `n_jobs=-1`, and keep the grid small. Evaluate final models on the full test set.

Tasks

(a) Data Preparation — 0.5 points
Report the number and percentage of fraudulent transactions, and explain why *accuracy* is not a suitable metric here.

(b) Building the Three Models — 1.5 points
Fit a **logistic regression** (with scaling), a **decision tree**, and a **random forest** (0.5 each). Default settings are fine; report test-set metrics.

(c) Dealing with Class Imbalance — 1.25 points

(i) **Choose and justify a metric.** Select **one primary metric** for this problem (e.g., F1, F_β, precision–recall AUC, ROC-AUC, or a cost-based metric). Explain why it is better than **at least two** alternatives other than accuracy, considering the class ratio, the cost of missed fraud versus false alarms, and whether the metric depends on a decision threshold. (0.75)

(ii) **Handle the imbalance.** Apply **one** strategy (class weighting, undersampling, SMOTE, or threshold tuning) to one model and compare it with the unadjusted version using your chosen metric. Resample inside the pipeline only; never choose a threshold on the test set. (0.5)

(d) Hyperparameter Optimization — 1 point
Tune **one** model (random forest recommended) over **at least two** hyperparameters with at least three values each, using **5-fold stratified cross-validation** and **your chosen metric from (c) as the scoring**. Include your imbalance strategy as a hyperparameter where possible (e.g., `class_weight`). Report the best setting, the mean and standard deviation of its CV score, and its test-set metrics. Briefly explain how the choice of scoring metric shapes which hyperparameters are selected, and why folds must be stratified and resampling done inside each fold.

(e) Model Comparison — 0.75 points
Summarize all models (default, imbalance-adjusted, and tuned) in one table using your chosen metric plus precision and recall. Which model would you deploy? Is the gain from tuning meaningful given the CV standard deviation? Comment on the false-positive / false-negative trade-off.

Submission Guidelines

Submit a runnable notebook (Python, R, or another programming language of your choice) through Blackboard (the link of your notebook, which stays in your Google drive or else where, is not accepted). Include sufficient markdown/comments to explain your analysis and decisions. Use appropriate labels, titles, units, and legends in visualizations. Report numerical results to three decimal places where appropriate, and report the random seed(s) you used.

Important: The goal is not to find the single highest score. Focus on a sound modeling process: use **proper train/test separation, cross-validation, appropriate metrics, and clear interpretation** to support your conclusions.